

Code Cultivation

Object-Oriented Garden Systems

Summary: Build a comprehensive digital garden ecosystem while discovering advanced Python concepts. Create tools to manage community gardens efficiently through data-driven approaches.

Version: 3.2

Contents

I	Foreword	2
II	AI Instructions	3
III	Introduction	5
IV	General Instructions	6
V	Exercise 0: Planting Your First Seed	7
VI	Exercise 1: Garden Data Organizer	9
VII	Exercise 2: Plant Growth Simulator	11
VIII	Exercise 3: Plant Factory	13
IX	Exercise 4: Garden Security System	15
X	Exercise 5: Specialized Plant Types	17
XI	Exercise 6: Garden Analytics	19
XII	Turn in and Submission	21

Chapter I

Foreword

In the digital age, even gardens need smart systems.

A wise gardener once said: "You can't plant flowers and then pull them up every day to see how the roots are doing." The same applies to code—sometimes the most beautiful growth happens when we trust the process and let our programs develop naturally, layer by layer.

Behind every thriving community garden lies a network of relationships. Plants don't grow in isolation; they form communities where each species contributes something unique while sharing common needs. Some plants protect others from pests, some fix nitrogen in the soil, and others provide shade for delicate seedlings. This interconnected web of mutual support mirrors how well-designed software systems work—individual components with distinct roles, working together harmoniously.

Just as a master gardener knows that healthy soil produces healthy plants, experienced programmers understand that well-structured code grows into robust applications. You'll discover that organizing your code is like planning a garden: some elements need protection, others need room to grow, and the best systems emerge when everything has its proper place and purpose.

In this project, you'll cultivate your programming skills while building tools that could genuinely help community gardens flourish. Every line of code you write is a seed planted in the digital soil of possibility.

Chapter II

AI Instructions

Context

During your learning journey, AI can assist with many different tasks. Take the time to explore the various capabilities of AI tools and how they can support your work. However, always approach them with caution and critically assess the results. Whether it's code, documentation, ideas, or technical explanations, you can never be completely sure that your question was well-formed or that the generated content is accurate. Your peers are a valuable resource to help you avoid mistakes and blind spots.

Main message

- Use AI to reduce repetitive or tedious tasks.
- Develop prompting skills — both coding and non-coding — that will benefit your future career.
- Learn how AI systems work to better anticipate and avoid common risks, biases, and ethical issues.
- Continue building both technical and power skills by working with your peers.
- Only use AI-generated content that you fully understand and can take responsibility for.

Learner rules:

- You should take the time to explore AI tools and understand how they work, so you can use them ethically and reduce potential biases.
- You should reflect on your problem before prompting — this helps you write clearer, more detailed, and more relevant prompts using accurate vocabulary.
- You should develop the habit of systematically checking, reviewing, questioning, and testing anything generated by AI.
- You should always seek peer review — don't rely solely on your own validation.

Phase outcomes:

- Develop both general-purpose and domain-specific prompting skills.
- Boost your productivity with effective use of AI tools.
- Continue strengthening computational thinking, problem-solving, adaptability, and collaboration.

Comments and examples:

- You'll regularly encounter situations — exams, evaluations, and more — where you must demonstrate real understanding. Be prepared, keep building both your technical and interpersonal skills.
- Explaining your reasoning and debating with peers often reveals gaps in your understanding. Make peer learning a priority.
- AI tools often lack your specific context and tend to provide generic responses. Your peers, who share your environment, can offer more relevant and accurate insights.
- Where AI tends to generate the most likely answer, your peers can provide alternative perspectives and valuable nuance. Rely on them as a quality checkpoint.

✓ Good practice:

> I ask AI: "How do I test a sorting function?" It gives me a few ideas. I try them out and review the results with a peer. We refine the approach together.

✗ Bad practice:

> I ask AI to write a whole function, copy-paste it into my project. During peer-evaluation, I can't explain what it does or why. I lose credibility — and I fail my project.

✓ Good practice:

> I use AI to help design a parser. Then I walk through the logic with a peer. We catch two bugs and rewrite it together — better, cleaner, and fully understood.

✗ Bad practice:

> I let Copilot generate my code for a key part of my project. It compiles, but I can't explain how it handles pipes. During the evaluation, I fail to justify and I fail my project.

Chapter III

Introduction

Welcome to CodeCultivation!

Building on your Python fundamentals from the first activity, you'll now tackle more complex programming challenges by creating a comprehensive garden data management system. This project introduces advanced concepts that make Python a powerful tool for modeling real-world systems.

You'll work on:

- Understanding how Python programs are structured and executed
- Organizing data using an object-oriented approach
- Creating reusable code components
- Building systems that can adapt and extend
- Protecting data integrity in collaborative environments
- Designing scalable software architectures

Each exercise builds on the previous ones, creating a complete digital garden ecosystem by the end.



IMPORTANT: This module starts with basic Python program structure, then progresses to Object-Oriented Programming. Each exercise should contain the requested definitions and any required code. You may include simple test code at the bottom of each file using if __name__ == "__main__": blocks for your own testing.

Chapter IV

General Instructions

- Your code must be written in Python 3.10+
- Your code must respect the `flake8` linter standards
- Each exercise must be in its own file and directory
- Use proper naming conventions: classes in PascalCase, functions and variables in snake_case
- All functions and methods must include type hints: use `mypy` to check your code
- You don't need to handle input validation unless explicitly mentioned
- Focus on demonstrating programming concepts clearly
- Your programs must always run without errors



Digital Garden Ecosystem Note: This project focuses on Python programming concepts, starting from basic program structure and progressing to Object-Oriented Programming. Each exercise introduces new features while building a cohesive garden management system. Later exercises will reuse concepts from earlier ones.



Object-Oriented Programming: Starting from Exercise 1, all exercises in this module require the use of classes. Python keywords such as 'class' and 'def' are fundamental language keywords and do not need to be listed in the authorized functions for each exercise.



For a nicer display, you can use the capitalize() method of an str (string) object. This will not be part of the "Authorized functions", but you can still use it if you want.

Chapter V

Exercise 0: Planting Your First Seed

	Exercise0
ft_garden_intro	
Directory: *ex*0/	
Files to Submit: `ft_garden_intro.py`	
Authorized: `print()`	

Before we can build complex garden management systems, you need to understand how Python programs work.
Every Python program needs a starting point - a place where execution begins. It's just like planting your first seed in a garden!

Your task is to create your very first Python program that displays information about a plant in your garden.

Requirements:

- Create a program that runs when executed directly using `python3 ft_garden_intro.py`
- Use the special `if __name__ == "__main__":` pattern.
- Store the following plant information in simple variables: name, height, and age.
- Display the plant information using `print()`
- Mimic the output display below; there will not be a strict output check during the evaluation.

Example:

```
$> python3 ft_garden_intro.py
=== Welcome to My Garden ===
Plant: Rose
Height: 25cm
Age: 30 days

=== End of Program ===
```



Understanding how programs start and execute is fundamental before moving to more complex concepts such as classes and other object-oriented features.

Note: Starting from this exercise and for all following exercises in this module (and future Python projects), you are allowed to use `if __name__ == "__main__":` blocks for testing your code. This pattern will not be explicitly displayed in the authorized functions, but it remains available during the entire project.



You must be able to explain why using the `if __name__ == "__main__":` line is important. This will lead you to the concept of imports, which will be part of an upcoming project. It is, however, important to introduce this good practice now.



Have you noticed that some Python files start with a special line beginning with #!? Research what this "shebang" line does and why it might be useful for making your scripts executable directly. The evaluation will ask you to update your script live to use this "shebang".



Chapter VI

Exercise 1: Garden Data Organizer

	Exercise1
ft_garden_data	
Directory: *ex*1/	
Files to Submit: `ft_garden_data.py`	
Authorized: `print()`	

The community garden needs to track multiple plants with their information. You need to store and display data for several plants efficiently.

Each plant has:

- A name
- Height in centimeters
- Age in days

Create a program that manages data for at least 3 different plants and displays their information in an organized way.

Required: You must create a `Plant` *class* that serves as a model for any plant, rather than handling each one individually. All plants will be described using the same *attributes*, or characteristics: a name, a height, and an age. For each plant, the class will be *instantiated*, then the attributes will be set to their specific values. You can use the name you want for each attribute.

You will create a `show()` *method* in the class to display the plant information.

Example:

```
$> python3 ft_garden_data.py
=== Garden Plant Registry ===
Rose: 25cm, 30 days old
Sunflower: 80cm, 45 days old
Cactus: 15cm, 120 days old
```



How are you currently storing plant information? There are multiple ways to do this. What challenges might arise with more plants? In the next exercises, you may eventually need to make your approach evolve.



If you have already worked with Python programming or object-oriented programming in the past, you will probably go directly for an advanced solution. In this case, you may find the next exercises a little bit redundant.

Chapter VII

Exercise 2: Plant Growth Simulator

	Exercise2
ft_plant_growth	
Directory: *ex2/*	
Files to Submit: `ft_plant_growth.py`	
Authorized: `print(), range(), round()`	

The garden manager wants to simulate plant growth over time. You need to track how plants change and provide *operations* to modify their state.

Requirements:

- Reuse your `Plant` *class* from Exercise 1
- Plants need to be able to `grow()` and `age()` on their own (i.e., these will be *methods* of the class)
- Simulate a week of growth for a plant, then access the data in the class to get the final height and display the total week increase.
- Consider how the plant should change over time when using `grow()` and `age()`. Different plants can evolve differently.

Your program should show a plant changing over time, and a summary at the end of the week. Think about giving your plants *behaviors* that will drive `grow()` differently.

Example:

```
$> python3 ft_plant_growth.py
=== Garden Plant Growth ===
Rose: 25.0cm, 30 days old
=== Day 1 ===
Rose: 25.8cm, 31 days old
=== Day 2 ===
Rose: 26.6cm, 32 days old
=== Day 3 ===
Rose: 27.4cm, 33 days old
=== Day 4 ===
Rose: 28.2cm, 34 days old
=== Day 5 ===
Rose: 29.0cm, 35 days old
=== Day 6 ===
Rose: 29.8cm, 36 days old
=== Day 7 ===
Rose: 30.6cm, 37 days old
Growth this week: 5.6cm
```

There are multiple possible approaches to handle this. You can implement it the way you want, adding attributes to the `Plant` class, parameters to the methods in the class, etc.

Chapter VIII

Exercise 3: Plant Factory

	Exercise3
ft_plant_factory	
Directory: *ex*3/	
Files to Submit: `ft_plant_factory.py`	
Authorized: `print(), range(), round()`	

The garden center needs to create many plants quickly using your `Plant` class with different starting values. You need to streamline the plant *creation* process: *instantiate* and *initialize* the class for a new plant at the same time.

Requirements:

- Plants need to be created directly with their initial information (name, starting height, starting age)
- Each plant should be ready to use immediately after *construction* (e.g., if you want to make it `grow()`)
- Create at least 5 different plants with varying characteristics
- Display all created plants in an organized format

Improve your `Plant` class from the previous exercises. Note that you may have already streamlined your plant creation process in Exercise 1 if you were already familiar with object-oriented programming.

Example:

```
$> python3 ft_plant_factory.py
=== Plant Factory Output ===
Created: Rose: 25.0cm, 30 days old
Created: Oak: 200.0cm, 365 days old
Created: Cactus: 5.0cm, 90 days old
Created: Sunflower: 80.0cm, 45 days old
Created: Fern: 15.0cm, 120 days old
```



You should be able to use the show() method unchanged to display the output above.

Chapter IX

Exercise 4: Garden Security System

	Exercise4
ft_garden_security	
Directory: *ex*4/	
Files to Submit: `ft_garden_security.py`	
Authorized: `print(), range(), round()`	

The garden manager is concerned about data integrity. Some volunteers accidentally corrupted plant data by setting invalid values (negative heights, impossible ages). You need to create a secure system that *protects* and *encapsulates* sensitive data.

Requirements:

- Improve your `Plant` class in order to protect its data from corruption
- Provide controlled ways to modify plant data: `set_height()`, `set_age()`
- Provide safe ways to access plant data: `get_height()`, `get_age()`
- Ensure plant height cannot be negative through validation
- Ensure plant age cannot be negative through validation
- Print error messages from the class when invalid values are provided, leaving data unchanged or creating the plant with default values
- Use *encapsulation*: prevent your class attributes from being used directly by using the *"protected" convention* (not the *mangling*)

Consider how you might create a system that *validates* data before storing it, ensuring data integrity.

Example:

```
$> python3 ft_garden_security.py
=== Garden Security System ===
Plant created: Rose: 15.0cm, 10 days old

Height updated: 25cm
Age updated: 30 days

Rose: Error, height can't be negative
Height update rejected
Rose: Error, age can't be negative
Age update rejected

Current state: Rose: 25.0cm, 30 days old
```

Chapter X

Exercise 5: Specialized Plant Types

	Exercise5
ft_plant_types	
Directory: *ex*5/	
Files to Submit: `ft_plant_types.py`	
Authorized: `super(), print(), range(), round()`	

The garden now needs to handle different types of plants: flowers, trees, and vegetables. Each type has unique characteristics but *inherits* common plant features from its *parent* category.

Requirements:

- Start with your `Plant` class from the previous exercise, which holds the common features (name, height, and age)
- Create specialized types: `Flower`, `Tree`, and `Vegetable`
- Each specialized type should *inherit* the basic plant features
- `Flower` needs: a color attribute and ability to `bloom()`
- `Tree` needs: a trunk_diameter attribute and the ability to `produce_shade()`
- `Vegetable` needs: a harvest_season and a nutritional_value attributes
- When creating specialized plants, call the parent methods from inside your new class using `super()`. It can be applied to any method, including `__init__()`
- A call to `show()` on a specialized class needs to print the standard `Plant` output and the extra characteristics of your specialized plant. Your method *override* can re-use the already existing code in the parent.
- Create at least one instance of each plant type; make the flower bloom; make the nutritional value start from 0, then increase when the vegetable's `age()` and `grow()` methods are called.

- Avoid duplicating common plant code across different specialized types.
- No need to validate the new attributes in the three new classes.

Example:

```
 $> python3 ft_plant_types.py
=== Garden Plant Types ===
=== Flower
Rose: 15.0cm, 10 days old
 Color: red
 Rose has not bloomed yet
[asking the rose to bloom]
Rose: 15.0cm, 10 days old
 Color: red
 Rose is blooming beautifully!

=== Tree
Oak: 200.0cm, 365 days old
 Trunk diameter: 5.0cm
[asking the oak to produce shade]
Tree Oak now produces a shade of 200.0cm long and 5.0cm wide.

=== Vegetable
Tomato: 5.0cm, 10 days old
 Harvest season: April
 Nutritional value: 0
[make tomato grow and age for 20 days]
Tomato: 47.0cm, 30 days old
 Harvest season: April
 Nutritional value: 20
```



How are you handling the common features shared by all plant types? It is advised to avoid repeating the same code in the different classes to handle these common features.

Chapter XI

Exercise 6: Garden Analytics

	Exercise6
ft_garden_analytics	
Directory: *ex*6/	
Files to Submit: `ft_garden_analytics.py`	
Authorized: `super()`, `print()`, `range()`, `round()`, `staticmethod()`, `classmethod()`	

Enhance your classes from the previous exercise to display analytics. You will need to handle complex data relationships, use *nested* components and *inheritance chains*.

Requirements:

- Create a *static method* for the `Plant` class that checks if a specific age given as a parameter is older than a year.
- Create a *class method* that allows you to create an "anonymous" plant directly when you do not yet have all the information.
- Create a `Seed` class that inherits from `Flower`, and holds the number of seeds once the flower has bloomed. The `show()` method must be improved accordingly.
- Each `Plant` has an internal system, implemented as a *nested* class, that holds statistical data: number of `grow()` calls, number of `age()` calls, number of `show()` calls. Encapsulation is required, as well as a display function.
- Trees need an extra piece of statistical data: number of `produce_shade()` calls.
- Finally, create a unique function, not part of any class, that displays statistics for any kind of plant.



`Note: the decorator syntax is also accepted for static and class methods`

Example:

```
$> python3 ft_garden_analytics.py
=== Garden statistics ===
=== Check year-old
Is 30 days more than a year? -> False
Is 400 days more than a year? -> True

=== Flower
Rose: 15.0cm, 10 days old
 Color: red
 Rose has not bloomed yet
[statistics for Rose]
Stats: 0 grow, 0 age, 1 show
[asking the rose to grow and bloom]
Rose: 23.0cm, 10 days old
 Color: red
 Rose is blooming beautifully!
[statistics for Rose]
Stats: 1 grow, 0 age, 2 show

=== Tree
Oak: 200.0cm, 365 days old
 Trunk diameter: 5.0cm
[statistics for Oak]
Stats: 0 grow, 0 age, 1 show
 0 shade
[asking the oak to produce shade]
Tree Oak now produces a shade of 200.0cm long and 5.0cm wide.
[statistics for Oak]
Stats: 0 grow, 0 age, 1 show
 1 shade

=== Seed
Sunflower: 80.0cm, 45 days old
 Color: yellow
 Sunflower has not bloomed yet
 Seeds: 0
[make sunflower grow, age and bloom]
Sunflower: 110.0cm, 65 days old
 Color: yellow
 Sunflower is blooming beautifully!
 Seeds: 42
[statistics for Sunflower]
Stats: 1 grow, 1 age, 2 show

=== Anonymous
Unknown plant: 0.0cm, 0 days old
[statistics for Unknown plant]
Stats: 0 grow, 0 age, 1 show
```



This exercise brings together all the programming patterns you have learned throughout the project. You will be evaluated on your understanding of how different components interact and organize themselves within a complex system.

Chapter XII

Turn in and Submission

Turn in your assignment in your `Git` repository as usual. Only the work inside your repository will be evaluated during the defense. Do not hesitate to double-check the names of your files to ensure they are correct. Also double-check Python standards and type hints using `flake8` and `mypy`.



During evaluation, you may be asked to explain programming concepts, demonstrate inheritance relationships, or extend your code with new functionality. Make sure you understand the principles behind your implementations.



You only need to submit the files requested by the subject of this project. Focus on clean, well-documented code that clearly demonstrates programming principles.